

Appendix

EEE3096S: Practical 1B Code

Phila Haanyama (HNYPHI001) and Lulama Lingela (LNLUL002)

File: Practical_1B/Core/Src/main.c

```
/* USER CODE BEGIN Header */
/**
 * ****
 * @file           : main.c
 * @brief          : Main program body
 * ****
 * @attention
 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * ****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdint.h>
#include "stm32f0xx.h"
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */
#define MAX_ITER 100
/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

/* USER CODE BEGIN PV */
//TODO: Define and initialise the global variables required
/*
    start_time
    end_time
    execution_time
    checksum: should be uint64_t
    initial width and height maybe or you might opt for an array??
*/
uint32_t start_time = 0;
uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;
```

```

int32_t height = 0;
int32_t width = 0;

const int image_dimensions[] = {128, 160, 192, 224, 256};

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int
max_iterations);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick.
    */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();

    /* USER CODE BEGIN 2 */

    for (int i = 0; i < 5; i++) {
        width = image_dimensions[i];
        height = image_dimensions[i];

        //TODO: Turn on LED 0 to signify the start of the operation
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

        // Fixed-point benchmark:
        //TODO: Record the start time
        start_time = HAL_GetTick();
    }
}

```

```

        //TODO: Call the Mandelbrot Function and store the output in the checksum
variable defined initially
        checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height,
MAX_ITER);
        //TODO: Record the end time
        end_time = HAL_GetTick();
        //TODO: Calculate the execution time
        execution_time = end_time - start_time;
        // Store or print execution_time & checksum

        // Double benchmark:
        //TODO: Record the start time
        start_time = HAL_GetTick();
        //TODO: Call the Mandelbrot Function and store the output in the checksum
variable defined initially
        checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
        //TODO: Record the end time
        end_time = HAL_GetTick();
        //TODO: Calculate the execution time
        execution_time = end_time - start_time;
        // Store or print execution_time & checksum

        //TODO: Turn on LED 1 to signify the end of the operation
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);

        //TODO: Hold the LEDs on for a 1s delay
        HAL_Delay(1000);

        //TODO: Turn off the LEDs
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
    }

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

/** Initializes the CPU, AHB and APB buses clocks
*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                             |RCC_CLOCKTYPE_PCLK1;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_RESET);

    /*Configure GPIO pins : PB0 PB1 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

//TODO: Mandelbrot using variable type integers and fixed point arithmetic
#define SCALE 4096 // scaling factor
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int
max_iterations){
    uint64_t mandelbrot_sum = 0;
    //TODO: Complete the function implementation
    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane
coordinates
            int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; //
making sure all numbers are integers not floats
            int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
                int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

```

```

        if(xi_sq + yi_sq <= 4 * SCALE){
            int32_t temp = (int32_t)(xi_sq - yi_sq);

            int64_t xy = ((int64_t)xi * yi) / SCALE;

            yi = (int32_t)(2*xy + y0);
            xi = temp + x0;

            i++;
        }
        else{
            break;
        }
    }

    mandelbrot_sum += i;
}
}
return mandelbrot_sum;
}

//TODO: Mandelbrot using variable type double
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
    uint64_t mandelbrot_sum = 0;
    //TODO: Complete the function implementation
    for (int y = 0; y < height; ++y) {
        double y0 = ((double)y / (double)height) * 2.0 - 1.0;      // map y
        -> [-1, +1]
        for (int x = 0; x < width; ++x) {
            double x0 = ((double)x / (double)width) * 3.5 - 2.5;  // map x
            -> [-2.5, 1.0]
            double xi = 0.0;
            double yi = 0.0;
            int i = 0;

            while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                double tmp = xi*xi - yi*yi + x0;
                yi = 2.0 * xi * yi + y0;
                xi = tmp;
                ++i;
            }
            mandelbrot_sum += (uint64_t)i;
        }
    }

    return mandelbrot_sum;
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)

```

```

    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line
    number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```